Athlete Nutritional & Training Recovery Dashboard

CCSICT
College of Computing Studies, Information and Communication technologies

By:
Group 6

To be submitted to:
Dan Joshua Sayo

Project Description

The Athlete Nutritional & Training Recovery Dashboard is a database-based system designed to help athletes, students, coaches, and administrators track and manage health and training information in one place. The system makes it easier to monitor nutrition, workout activities, and recovery progress.

Users can record and manage:

- -Nutrition logs and calorie intake
- -Training sessions and workout activities
- -Recovery logs and rest records
- -Meal types and other fitness-related information

The system also shows simple graphs and reports to help users see nutrition trends, training progress, and recovery performance. These features help users better understand their health and fitness status.

The dashboard is built using Python, Flask, Gunicorn, and PyMySQL for fast and smooth performance. It supports both MySQL and MariaDB databases and allows administrators to change the SQL server IP or host if needed.

Overall, the system is designed to be simple, fast, and easy to use while providing the important tools needed for athlete nutrition and recovery monitoring.

Objective of the Sytem

The main goal of this project is to build a reliable and easy-to-use dashboard that helps athletes and coaches track essential performance data. The specific objectives are:

Centralize Data: To create a single database for nutrition, training, and recovery logs so all information is organized in one place.

Track Daily Progress: To give users a fast way to record and monitor their calories, macros, and workout sessions daily.

Analyze Recovery: To help athletes see the connection between their training intensity and how well they are recovering, making it easier to avoid burnout.

Ensure Data Security: To build a secure login system that separates Admin and User roles, keeping everyone’s personal information private.

Optimize Performance: To use a Flask and Gunicorn backend to ensure the app stays fast and responsive when handling data requests.

Maintain Flexibility: To allow the system to connect easily to different SQL servers or hosts for simple deployment.

Business Rules

The following business rules define the constraints, relationships, and operational logic of the Athlete Management System. These rules ensure data integrity, consistency, and proper system behavior across all modules.

User and Authentication Rules

1. Each user (athlete or system user) must have a unique username.
2. Each user account is identified by a unique athlete_id.
3. Passwords are required for authentication and must be securely stored.
4. Users are assigned roles such as admin, user, or owner, which determine system access permissions.
5. Only authorized roles (admin/owner) may perform administrative operations if enforced by the application.

Athlete Information Rules

1. Each athlete must have a complete profile including name, age, sex, weight, and height.
2. Sex values are restricted to: Male, Female, or Other only.
3. Age, weight, and height must be numeric values when provided.
4. Each athlete record must be unique and stored using athlete_id.

Nutrition Log Rules

1. Each nutrition log entry must be associated with exactly one athlete using athlete_id.
2. Calories are required and must be a positive integer.
3. Macronutrient values (protein, carbs, fats) are stored as decimal values for precision.
4. Each nutrition entry must include a valid log_date.
5. If an athlete is deleted, all related nutrition logs are automatically removed (ON DELETE CASCADE).

Training Session Rules

1. Each training session must be linked to one valid athlete.
2. Session duration (duration_minutes) must be greater than zero.
3. Intensity level is restricted to: Low, Medium, or High.
4. Calories burned may be null if not recorded but must be numeric when provided.
5. Each session must include a valid session_date.

Recovery Log Rules

1. Each recovery log must be linked to a valid athlete.
2. Sleep hours may include decimal values for precision.
3. Soreness level, stress level, and recovery score must be numeric values within a valid range defined by the system.
4. Each recovery entry must include a valid log_date.

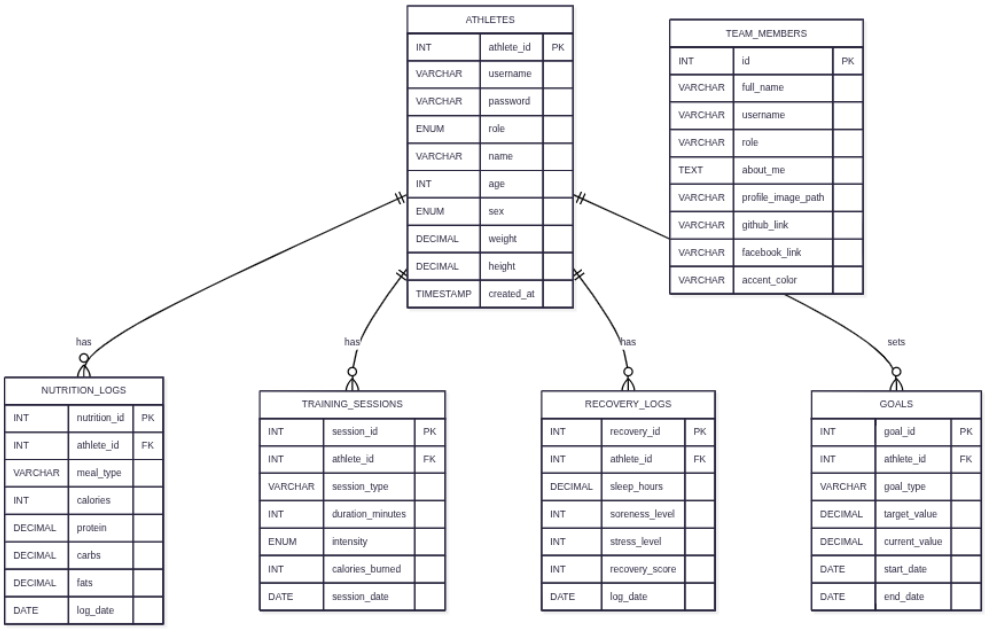
Goal Tracking Rules

5. Each goal must be associated with exactly one athlete.
6. Goal types must describe a measurable objective (e.g., strength, weight, endurance).
7. Target values and current values must be numeric and comparable.
8. Each goal includes a defined start date and optional end date.

Team Member Rules

9. Each team member entry must include a full name and assigned role.
10. Each team member may customize their profile appearance using an accent color.
11. Team member records are independent of athlete records and are used for system presentation only.

ER diagram



The ER diagram illustrates the structure of the Athlete Management System database, showing its entities, attributes, and relationships. Each entity represents a real-world object within the system, while relationships define how these entities interact with each other.

ATHLETES Entity

- The ATHLETES table is the central entity of the system. It stores core user information such as username, password, role, name, age, sex, weight, height, and account creation timestamp.
- Each athlete is uniquely identified by athlete_id, which serves as the primary key.

NUTRITION_LOGS Entity

- The NUTRITION_LOGS entity stores dietary information for each athlete, including meal type, calories, and macronutrients (protein, carbs, fats) along with the log date.
- It uses athlete_id as a foreign key to link each nutrition entry to a specific athlete.
- 🏋️ TRAINING_SESSIONS Entity
- The TRAINING_SESSIONS entity records workout activities performed by athletes. It includes session type, duration, intensity level, calories burned, and session date.
- Each record is associated with an athlete through the athlete_id foreign key.

RECOVERY_LOGS Entity

- The RECOVERY_LOGS entity tracks athlete recovery data such as sleep hours, soreness level, stress level, recovery score, and log date.
- It is linked to the ATHLETES table using athlete_id, ensuring that each recovery entry belongs to a specific athlete.

GOALS Entity

- The GOALS entity represents fitness objectives set by athletes. It includes goal type, target value, current progress, and timeline (start and end dates).
- Each goal is associated with an athlete through the athlete_id foreign key.

TEAM_MEMBERS Entity

- The TEAM_MEMBERS entity stores information about system contributors such as developers or collaborators. It includes personal details, role description, and social links.
- Unlike other entities, it is independent and not directly linked to athletes

Relationships

- The ER diagram shows a one-to-many relationship between ATHLETES and multiple dependent entities:
  - One athlete can have many nutrition logs
  - One athlete can have many training sessions
  - One athlete can have many recovery logs
  - One athlete can have many goals

These relationships are enforced using foreign keys (athlete_id) in each related table. This ensures data integrity and allows efficient tracking of each athlete's performance, nutrition, recovery, and progress over time.

Normalization Summary

The athlete management system was designed to store athlete profiles, nutrition tracking, training sessions, recovery monitoring, goals, and team member information. The database was normalized from Unnormalized Form (UNF) to Third Normal Form (3NF) to eliminate redundancy, ensure data integrity, and improve scalability.

Unnormalized form

- athlete_id username password role name age sex weight height meal_type calories session_type sleep_hours goal_type
- Problems:
  - Repeated athlete data for every meal/session/log
  - Mixed unrelated data (nutrition + training + recovery)
  - Hard to update (changing name requires updating many rows)
  - High redundancy and inconsistency risk

First normal Form

- Each column must contain atomic (single) values
- No repeating groups

- Before:
  - Athlete | meals
- After
  - | nutrition_id | athlete_id | meal_type | calories |
- Result:
  - Each table contains atomic values
  - No multi-value fields exist
  - Data is properly split into logical entities

Second normal Form

- Must be in 1NF
- Remove partial dependency (data depending only on part of a composite key)
- Problem:
  - athlete_name depends only on athlete_id (not full key)
- Correct one:
  - athletes
    - | athlete_id | username | name | age | sex | weight | height |
  - training_sessions
    - | session_id | athlete_id | session_type | duration_minutes | intensity |
- Result:
  - Athlete data is stored once only
  - Session data depends fully on session_id + athlete_id relationship
  - No partial dependency exists

Third normal form

- Must be in 2NF
- Remove transitive dependency (non-key fields must not depend on other non-key fields)
- Example:
  - athletes (
    athlete_id,
    username,
    password,
    role,
    name,
    age,
    sex,

weight,
height

)

- TRAINING_SESSIONS(session_type, duration, intensity, calories_burned)
- goals (goal_type, target_value, current_value)
- team_members (full_name, username, role, about_me, links)

- Result:
  - This table is in **3NF** (no derived attributes stored)

System Features

The Athlete Management System is a web-based application designed to efficiently manage athlete performance, nutrition, and recovery data. The system integrates authentication, data tracking, visualization, and database flexibility to support both usability and scalability.

Authentication and Role-Based Access Control

- The system includes a secure login mechanism that separates users based on roles such as admin and standard user. Each account is protected using password authentication, ensuring that only authorized users can access or modify system data. Role-based access control restricts administrative functions to authorized personnel only.

Athlete Performance Data Management

- The system allows users to record and manage comprehensive athlete data, including:
  - Calorie intake and nutrition logs
  - Training session details
  - Macronutrient tracking (protein, carbohydrates, fats)
  - Meal types and dietary records
  - Recovery-related metrics such as sleep, stress, and soreness level.
- This enables centralized tracking of athlete health and performance over time.

Data Visualization and Analytics

- The system provides simple graphical representations to support performance analysis. These include:
  - Nutrition trend visualization over time
  - Training performance trends
  - Recovery status analysis
- These visual tools help users easily interpret progress and identify areas for improvement.

Backend Performance Optimization

- The application is optimized for efficient request handling using:

    - Flask as the core web framework
    - PyMySQL for database connectivity
    - Gunicorn as a production-ready WSGI server
- This combination ensures fast query execution, improved concurrency, and stable server performance under load.

Database Flexibility and Configuration

- The system supports dynamic database configuration, allowing users to modify the SQL server host or IP address. This enables seamless switching between local, remote, or cloud-based database environments.

Database Compatibility

- The system is designed to support both MySQL and MariaDB databases, ensuring flexibility and compatibility across different deployment environments without requiring major code changes.

Summary

The Athlete Management System was designed as a structured and scalable web-based application for tracking and managing athlete performance data. It integrates key modules such as authentication, nutrition tracking, training session monitoring, recovery analysis, and goal setting into a unified system.

The database was carefully designed following normalization principles up to Third Normal Form (3NF), ensuring minimal redundancy, improved data integrity, and efficient relationships between entities. The Entity-Relationship Diagram (ERD) further illustrates how the system is structured around a central ATHLETES entity connected to multiple activity-based records through well-defined foreign key relationships.

In addition, the system implements role-based authentication to ensure secure access, while also providing data visualization features that help users analyze trends in nutrition, training, and recovery performance. Backend optimization using Flask, PyMySQL, and Gunicorn ensures fast and reliable system performance, while database flexibility allows support for both MySQL and MariaDB environments.

Overall, the system demonstrates a complete and practical implementation of database design principles, web development practices, and performance optimization techniques, making it a functional and extensible solution for athlete data management.